

Finite-Dimensional Type I von Neumann Algebras in PyTorch: A GPU-Accelerated Framework for Random Block-Diagonal Operators

Irina Nikolaeva

Andrej Novikov

Abstract

We present `torch_vn_algebra`, an open-source Python library built on PyTorch for numerical experiments with finite-dimensional Type I von Neumann algebras (direct sums of matrix algebras). The library provides:

- a compact batched tensor representation $(B, C, k_{\max}, k_{\max})$ that handles both Monte Carlo samples and multiple direct summands;
- lazy evaluation of operators to avoid unnecessary memory allocation;
- generation of random operators with arbitrary eigenvalue distributions (user-provided samplers) and several unitary ensembles (Haar on $U(n)$ and $O(n)$, $SU(n)$ and $SO(n)$, COE, CSE, diagonal phases);
- functional calculus via SVD (absolute value, square root, inverse, entropy) and a hybrid method for extreme eigenvalues (exact diagonalisation for blocks of size $k_c \leq 256$, otherwise shifted power iteration);
- three trace functionals (blunt, normalised subspace trace, and the von Neumann tracial state);
- GPU-accelerated batched linear algebra for moderate-scale Monte Carlo studies (e.g. 2×10^4 samples of operators with up to 32 blocks).

The library is validated against analytical expectations (Haar moments, trace properties). Performance benchmarks on a Tesla P100 GPU are presented and discussed, and three Monte Carlo experiments on trace inequalities are reported. Limitations and future work are outlined. The code is open-source.

1 Introduction

Finite-dimensional Type I von Neumann algebras,

$$\mathcal{M} = \bigoplus_{c=1}^C M_{n_c}(\mathbb{C}),$$

naturally appear in quantum mechanics with superselection rules, decoherence, and in random matrix theory when symmetries or constraints force operators into block-diagonal form. Monte Carlo simulations of such systems – averaging over many random operators with controlled spectral properties – require efficient numerical tools that can exploit modern hardware. General-purpose libraries (NumPy/SciPy, QuTiP, Qiskit) do not provide a native batched representation of block-diagonal (direct-sum) operators together with arbitrary prescribed spectra and GPU execution.

`torch_vn_algebra` fills this gap by combining a compact tensor representation that batches both Monte Carlo samples and direct summands with padding to a common $k_{\max}$; lazy evaluation (operators are built from generators and materialised only when needed); flexible random operators (any eigenvalue distribution via a user callable) with several unitary ensembles; batched functional calculus with hybrid eigenvalue extraction; three trace functionals Tr_{blunt} , Tr_{norm} and the von Neumann tracial state τ_{vN} ; and full integration with PyTorch’s GPU backend.

The library builds upon the theoretical framework developed in [?, ?], where the Michelson contrast was generalised to operators in von Neumann algebras. The present work provides a scalable GPU implementation of these concepts for Monte Carlo studies.

The paper is organised as follows. Section ?? recalls necessary definitions. Section ?? describes the implementation. Section ?? presents validation and performance benchmarks. Section ?? gives Monte Carlo experiments. Section ?? discusses limitations and future work. Section ?? concludes.

2 Mathematical background

A finite-dimensional Type I von Neumann algebra is $\mathcal{M} = \bigoplus_{c=1}^C M_{n_c}(\mathbb{C})$ acting on $\mathcal{H} = \bigoplus_{c=1}^C \mathbb{C}^{k_c}$ with $k_c \leq n_c$. Operators are block-diagonal: $A = \bigoplus_c A_c$. Three trace functionals are relevant:

$$\text{Tr}_{\text{blunt}}(A) = \sum_{c=1}^C \text{Tr}(A_c), \quad \text{Tr}_{\text{norm}}(A) = \sum_{c=1}^C \frac{1}{k_c} \text{Tr}(A_c^{(k_c)}), \quad \tau_{\text{vN}}(A) = \frac{1}{C} \sum_{c=1}^C \frac{1}{k_c} \text{Tr}(A_c^{(k_c)}),$$

where $A_c^{(k_c)}$ is the restriction of A_c to the k_c -dimensional subspace. τ_{vN} is a faithful normal tracial state. For $A \in \mathcal{M}$ we write $|A| = (A^*A)^{1/2}$. For a density matrix ρ ($\rho \geq 0$, $\text{Tr} \rho = 1$), the von Neumann entropy is $S(\rho) = -\text{Tr}(\rho \log \rho)$. For a positive operator A , define its Michelson contrast as

$$\Delta(A) = \frac{\lambda_{\max}(A) - \lambda_{\min}(A)}{\lambda_{\max}(A) + \lambda_{\min}(A)},$$

where the extreme eigenvalues are taken over the whole algebra, i.e. over all blocks. $\Delta(A) \in [0, 1]$, and $\Delta(A) = 0$ if and only if A is a scalar multiple of the identity.

The theory of noncommutative integration associates to a von Neumann algebra $\mathcal{M}$ equipped with a faithful normal trace τ a family of noncommutative L^p -spaces. In the finite-dimensional Type I case, the L^1 -space is simply $\mathcal{M}$ with the norm $\|A\|_1 = \tau(|A|)$. For a positive operator X affiliated with $\mathcal{M}$, one can define its L^1 -norm as $\tau(X)$ when finite [?].

3 Implementation

Operators are stored as a 4-D tensor $(B, C, k_{\max}, k_{\max})$ with the active block of size $k_c \times k_c$ in the top-left corner; the remaining entries are zero padding. The `Operator` class uses lazy generation: the matrix is materialised only when accessed. The auxiliary class `HilbertSpace` provides inner product, norm, and orthonormal bases (standard, random, Haar), supporting embedding and restriction of vectors and operators.

Random operators. Random operator generation is performed via `operator_from_eigenvalues`. The user supplies a callable `eigenvalue_sampler(dim)` returning either one spectrum of length `dim` or a batch of spectra of shape (B, dim) . For each channel a batch of random unitaries U is drawn from the chosen ensemble and $A_c = U \text{diag}(\lambda) U^*$ is formed (and symmetrised when the spectrum is real). Properties (self-adjointness, positivity, etc.) are inferred from the spectrum; forced properties are validated when the matrix is materialised, with tolerances relative to the working precision.

Unitary ensembles. Haar-distributed unitaries are sampled by the QR decomposition of a Ginibre matrix with i.i.d. circular complex (or real) standard Gaussian entries, followed by fixing the phases of the diagonal of R [?]. $SU(n)$ is obtained by multiplying a Haar unitary by $\det(U)^{-1/n}$ and $SO(n)$ by flipping the sign of one column when $\det = -1$; both maps push Haar measure forward to Haar measure. The circular orthogonal and symplectic ensembles are $S = W^T W$ and $S = W^R W$, $W^R = JW^T J^T$, with W Haar on $U(n)$ [?]. All samplers are batched.

Extreme eigenvalues. For self-adjoint operators, $\lambda_{\max}$ and $\lambda_{\min}$ are obtained by exact diagonalisation (`torch.linalg.eigvalsh`) when all blocks satisfy $k_c \leq 256$ (threshold `exact_eig_max_dim`). For larger blocks a two-stage power iteration is used in double precision: first the eigenvalue μ of largest modulus is found, then the dominant eigenvalue of $\mu I - A$, which gives the opposite end of the spectrum (at most 5000 iterations, stopping when the Rayleigh quotient changes by less than 10^{-12}). The operator norm uses a shorter single-precision power iteration on $A^* A$ (at most 100 iterations, tolerance 10^{-8}).

Functional calculus and traces. $|A|$, $\sqrt{A}$ (for $A \geq 0$), the (pseudo-)inverse A^{-1} and $\text{Tr}(A \log A)$ are computed via batched SVD, $A = W \Sigma V^*$, $|A| = V \Sigma V^*$. The three traces are implemented as `Tr_blunt`, `Tr_norm` and `tau_vN`; the Michelson contrast `michelson_contrast` returns $\Delta(A)$ for every sample of the batch.

4 Validation and performance benchmarks

4.1 Validation

All validation runs are reproduced by `scripts/validation.py`.

Haar measure. For U Haar-distributed on $U(n)$ one has $\mathbb{E}|U_{11}|^2 = 1/n$ and, by invariance under $U \mapsto e^{i\varphi} U$, $\mathbb{E}[U_{11}^2] = 0$. The second identity fails for non-circular Gaussian input and is therefore a useful check of the sampler. Table ?? shows the results for $n = 2, 4, 8, 16, 32$ using 10 000 samples generated by the library. All relative errors are within about two standard errors of the mean.

n	$1/n$	relative error of $\mathbb{E} U_{11} ^2$	relative standard error	$ \mathbb{E}[U_{11}^2] $
2	0.5	9.3×10^{-4}	5.8×10^{-3}	3.3×10^{-3}
4	0.25	5.1×10^{-3}	7.8×10^{-3}	3.5×10^{-3}
8	0.125	4.7×10^{-3}	8.9×10^{-3}	6.6×10^{-4}
16	0.0625	1.9×10^{-2}	9.4×10^{-3}	3.0×10^{-4}
32	0.03125	2.0×10^{-2}	9.6×10^{-3}	2.7×10^{-4}

Table 1: Haar measure on $U(n)$, 10 000 samples.

Power iteration. For a 3×3 matrix with eigenvalues $[1.0, 0.99, 0.98]$ (gap 0.01), the power iteration needed 699 iterations before successive Rayleigh quotients differed by less than 10^{-8} , compared to 15 iterations for the well-separated spectrum $[2, 1, 0.5]$. Note that for the small gap the actual error at that point was 4.9×10^{-7} , i.e. a stopping rule on successive differences overestimates the accuracy when the gap is small. This is why the library uses exact diagonalisation up to $k_c = 256$.

Square root. We generated 1000 random 100×100 positive matrices (eigenvalues uniform in $[0, 1]$) and computed $B = \sqrt{A}$ via SVD. The relative error $\|B^2 - A\|_F / \|A\|_F$ was 2.0×10^{-6} on average (maximum 2.4×10^{-6}) in the library’s default single precision and 4.0×10^{-15} (maximum 4.9×10^{-15}) in double precision, i.e. at the level of the respective machine epsilon.

4.2 Benchmark setup and speedup results

All benchmarks were run on an NVIDIA Tesla P100 (16 GB) and an Intel Xeon E5-2650 v4 @ 2.2 GHz (12 cores) with `scripts/benchmark.py`. Each test input is a tensor of shape (C, k, k) , i.e. the channel axis plays the role of the batch, with $k \in \{2, 4, \dots, 64\}$ and $C \in \{1, 2, 4, \dots, 1024\}$; every configuration was timed with three random seeds and ten repetitions. Speedup is defined as $t_{\text{CPU}}/t_{\text{GPU}}$ and includes the final device synchronisation but not the host-to-device transfer. The benchmark times the batched PyTorch kernels underlying the library operations (trace, addition, multiplication, inverse, SVD, symmetric eigensolver, norms) on Gaussian random matrices; the CPU used PyTorch’s default thread count.

Table ?? shows the speedup for the inverse and for matrix multiplication. A second, independent run with the same configuration (stored in the repository as `results/benchmark/run_b`) agrees with the first within a median factor of 1.12, but single configurations may differ considerably, so the numbers should be read as orders of magnitude. For a single channel the CPU is always faster (speedup 0.3–0.7), because kernel launch overhead dominates. The GPU pays off from about $C \geq 64$ blocks of size $k \geq 8$, with speedups of 15–135 at $C = 1024$. For the SVD and the symmetric eigensolver the GPU becomes *slower* than the CPU at $k = 64$ (speedup 0.2–0.6 for all C), which marks the transition from the batched small-matrix GPU solvers to the general ones; this is the main practical limitation for larger blocks (Section ??).

k	inverse			multiplication		
	$C = 1$	$C = 64$	$C = 1024$	$C = 1$	$C = 64$	$C = 1024$
2	0.3	1.6 (1.0)	11.3	0.4	0.5 (0.5)	0.5
4	0.3	1.5 (1.2)	15.5	0.4	0.5 (0.4)	1.0
8	0.3	25.1 (16.0)	134.8	0.5	2.5 (2.0)	27.2
16	0.4	21.0 (19.1)	50.6	0.5	2.8 (2.2)	34.8
32	0.3	34.7 (40.5)	49.0	0.6	6.0 (4.1)	58.3
64	0.5	17.9 (10.7)	18.9	0.7	11.0 (10.6)	138.4

Table 2: Mean speedup $t_{\text{CPU}}/t_{\text{GPU}}$ for C blocks of size k (first run; values of the second run in parentheses).

5 Monte Carlo experiments on trace inequalities

We perform three Monte Carlo experiments that mirror earlier NumPy code and illustrate the library’s capabilities. All experiments are reproduced by `scripts/experiment.py`. For each block size $k \in \{2, 16\}$ and number of channels $C \in \{1, 2, 16, 32\}$ (all $k_c = k$) we generate 20 000 random operator pairs in the real algebra $\mathcal{M} = \bigoplus_{c=1}^C M_k(\mathbb{R})$. Operators are constructed via the spectral theorem: in every block the largest eigenvalue is 1, the smallest is $(1 - \delta)/(1 + \delta)$ with δ uniform in $[0, 1]$, and the remaining eigenvalues are uniform in between; the eigenvectors are Haar-random orthogonal matrices, independent for every block and sample. Throughout this section $\text{Tr} = \text{Tr}_{\text{blunt}}$, and “ U unitary in $\mathcal{M}$ ” means $U = \bigoplus_c U_c$ with independent Haar-random orthogonal U_c .

Table ?? summarises the sign of the quantity z in each experiment; the figures show scatter plots of z against the contrasts. Raw samples, all plots, and the summary table are available in the repository ([results/experiments](#)).

k	C	Exp. 1		Exp. 2		Exp. 3	
		$z > 0, \%$	$\max z$	$z > 0, \%$	$z < 0, \%$	$z < 0, \%$	$\min z$
2	1	7.0	0.227	52.9	46.6	0.0	-8.34e-07
2	2	0.7	0.328	54.2	45.8	0.0	-9.54e-07
2	16	0.0	-4.37	62.4	37.6	0.0	0.00115
2	32	0.0	-15.9	68.2	31.8	0.0	0.0764
16	1	0.0	-2.18	57.4	41.9	0.0	-7.63e-06
16	2	0.0	-5.73	63.0	36.9	0.0	1.72e-05
16	16	0.0	-89	86.4	13.6	0.0	1.83
16	32	0.0	-193	94.1	5.9	0.0	6.11

Table 3: Sign of z over 20 000 samples per configuration ($|z| \leq 10^{-4}$ counted as zero).

5.1 Experiment 1 (positive X , positive Y , unitary U)

Both X and Y are positive operators and U is a random unitary in $\mathcal{M}$. The quantity of interest is

$$z = |\text{Tr}(XUY)| - \text{Tr}(XY).$$

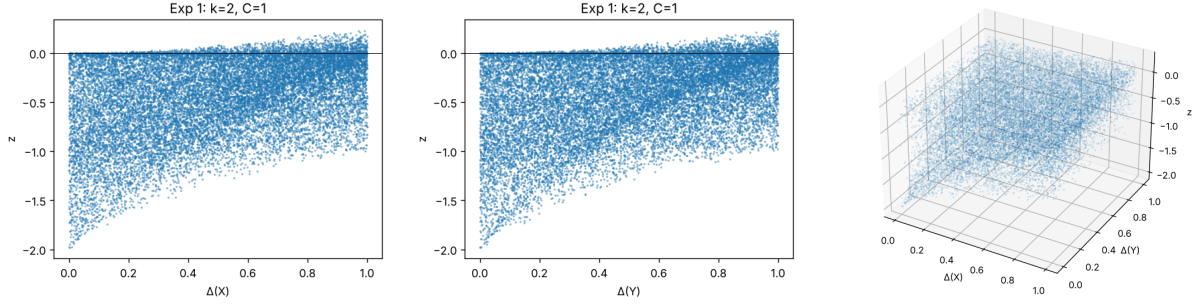
The inequality $|\text{Tr}(XUY)| \leq \text{Tr}(XY)$ holds for all unitaries U when X and Y commute, but not in general: writing the polar decomposition $YX = W|YX|$ with W unitary in $\mathcal{M}$ and choosing $U = W^*$ gives $\text{Tr}(XUY) = \text{Tr}(W^*YX) = \|YX\|_1 \geq \text{Tr}(YX) = \text{Tr}(XY)$, with strict inequality whenever YX is not positive. Random unitaries come close to this extremal choice only rarely, and increasingly so as k and C grow. In the simulations $z > 0$ occurs in 7.0% of the samples for $k = 2$, $C = 1$ and in 0.7% for $k = 2$, $C = 2$, but in none of the samples with $k = 16$ or $C \geq 16$. Positive values occur only away from scalar operators: for $k = 2$, $C = 1$ the largest z over samples with $\Delta(X), \Delta(Y) < 0.1$ is -4×10^{-4} , and the size of the violations grows with $\Delta(X)$ (Figure ??). The contrasts $\Delta(X)$ and $\Delta(Y)$ are computed from the global extreme eigenvalues.

Listing 1: Experiment 1 implementation

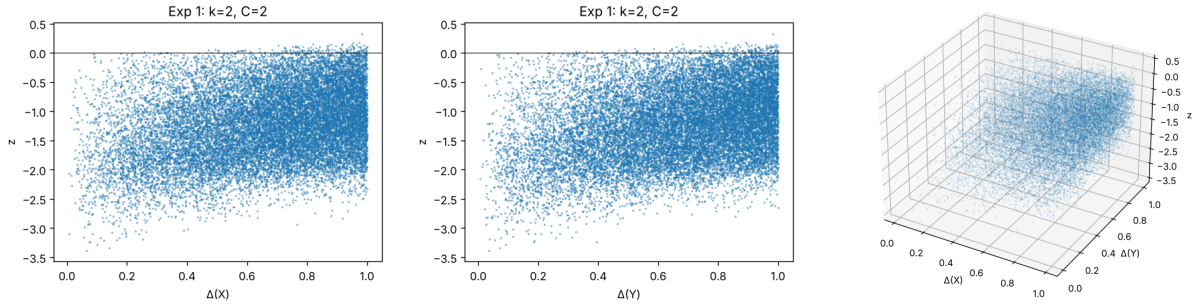
```

1 def experiment1(alg, batch_size, num_batches, device):
2     pos = positive_sampler(batch_size, device)
3     out = []
4     for _ in range(num_batches):
5         X = alg.operator_from_eigenvalues(pos, batch_size=batch_size,
6                                         force_positive=True, force_self_adjoint=True)
7         Y = alg.operator_from_eigenvalues(pos, batch_size=batch_size,
8                                         force_positive=True, force_self_adjoint=True)
9         U = alg.random_unitary_operator(batch_size)
10        z = (X @ U @ Y).trace.abs() - (X @ Y).trace.real
11        out.append((X.michelson_contrast, Y.michelson_contrast, z))
12    return collect(out)

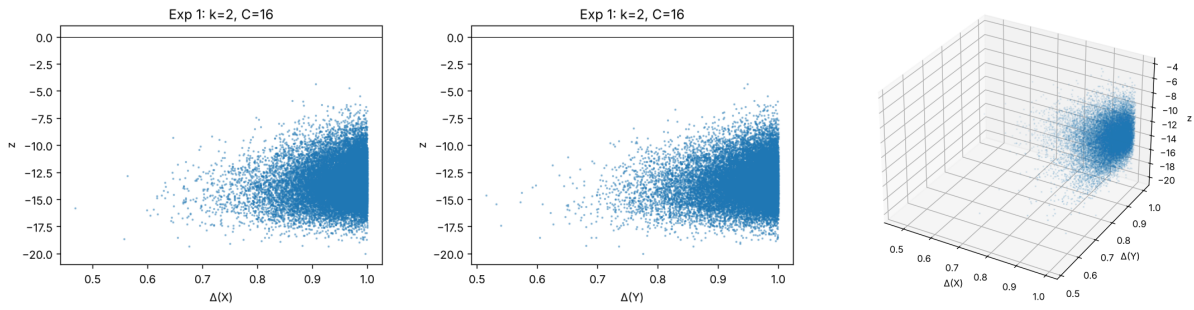
```



(a) $k = 2, C = 1$

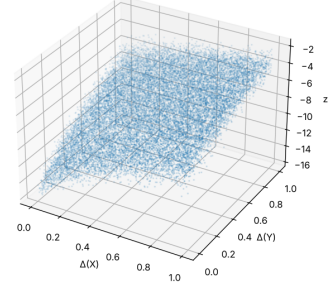
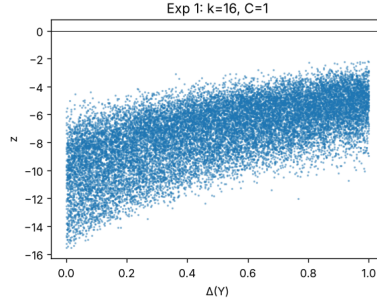
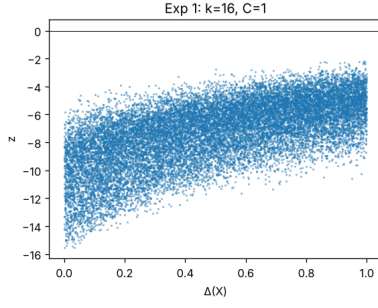


(b) $k = 2, C = 2$

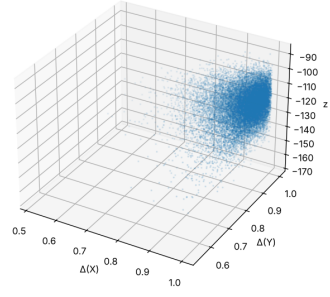
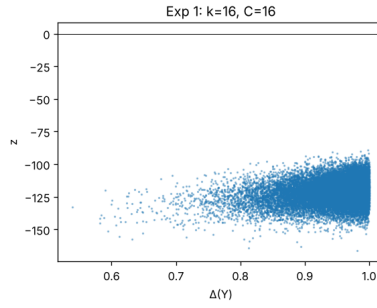
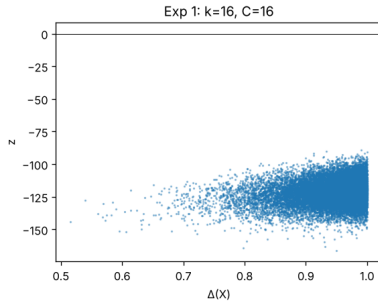


(c) $k = 2, C = 16$

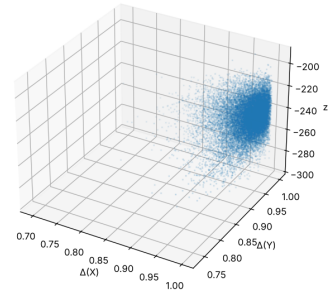
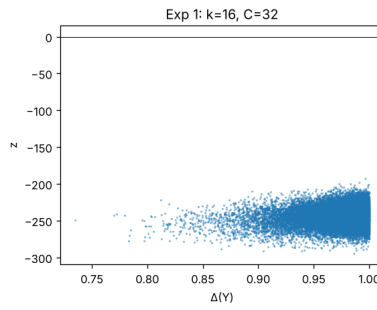
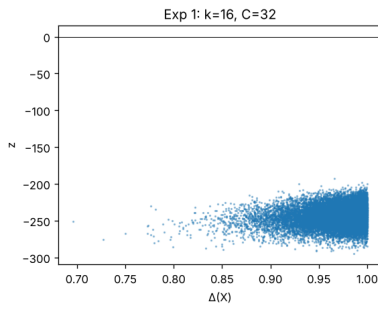
Figure 1: Experiment 1: small dimension ($k = 2$). Left: z vs. $\Delta(X)$; middle: z vs. $\Delta(Y)$; right: both.



(a) $k = 16, C = 1$



(b) $k = 16, C = 16$



(c) $k = 16, C = 32$

Figure 2: Experiment 1: intermediate dimension ($k = 16$). Left: z vs. $\Delta(X)$; middle: z vs. $\Delta(Y)$; right: both.

5.2 Experiment 2 (non-Hermitian X , non-Hermitian Y)

Both X and Y are made non-Hermitian by multiplying positive operators with random unitaries: $X = UX_0$, $Y = VY_0$, so that $|X| = X_0$ and $|Y| = Y_0$. The quantity of interest is

$$z = \text{Tr}|XY| - \text{Tr}(|X||Y|) = \|X_0VY_0\|_1 - \text{Tr}(X_0Y_0).$$

z takes both signs: for $V = I$ one has $z = \|X_0Y_0\|_1 - \text{Tr}(X_0Y_0) \geq 0$, while for V far from the identity $\|X_0VY_0\|_1$ can be much smaller than $\text{Tr}(X_0Y_0)$. In the simulations $z > 0$ in 53–94% of the samples, the share growing with k and C . The contrasts are computed from $|X| = X_0$ and $|Y| = Y_0$, i.e. from the singular values of X and Y .

Listing 2: Experiment 2 implementation

```

1 def experiment2(alg, batch_size, num_batches, device):
2     pos = positive_sampler(batch_size, device)
3     out = []
4     for _ in range(num_batches):
5         X0 = alg.operator_from_eigenvalues(pos, batch_size=batch_size,
6                                           force_positive=True, force_self_adjoint=True)
7         Y0 = alg.operator_from_eigenvalues(pos, batch_size=batch_size,
8                                           force_positive=True, force_self_adjoint=True)
9         X = alg.random_unitary_operator(batch_size) @ X0
10        Y = alg.random_unitary_operator(batch_size) @ Y0
11        z = (X @ Y).abs().trace.real - (X.abs() @ Y.abs()).trace.real
12        out.append((X0.michelson_contrast, Y0.michelson_contrast, z))
13    return collect(out)

```

5.3 Experiment 3 (self-adjoint X , positive Y)

Here X is self-adjoint (eigenvalues with independent random signs) and Y is positive. The quantity of interest is

$$z = \text{Tr}(Y|X|Y) - \text{Tr}|YXY|.$$

Proposition 1. *For $X = X^*$ and $Y \geq 0$ in $\mathcal{M}$ one has $z \geq 0$. If Y commutes with X (in particular, if Y is central), then $z = 0$.*

Proof. Write $X = X_+ - X_-$ with $X_{\pm} \geq 0$, $|X| = X_+ + X_-$. By the triangle inequality for the trace norm, $\text{Tr}|YXY| = \|YX_+Y - YX_-Y\|_1 \leq \text{Tr}(YX_+Y) + \text{Tr}(YX_-Y) = \text{Tr}(Y|X|Y)$. If $YX = XY$, then $YXY = Y^2X$ and $|YXY| = Y^2|X| = Y|X|Y$. $\square$

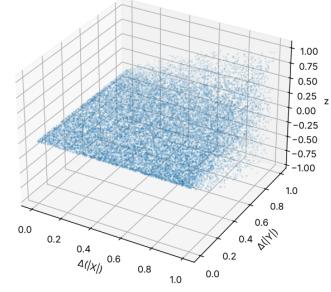
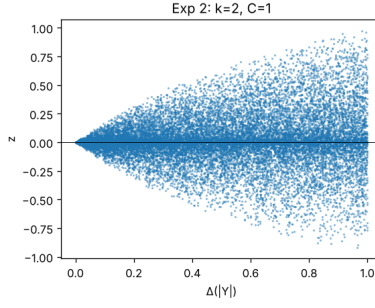
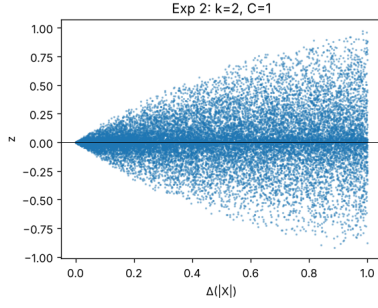
The simulations agree with the proposition: no sample has $z < 0$ beyond round-off ($|z| \leq 10^{-5}$). $z = 0$ also occurs whenever all eigenvalues of X have the same sign, which for $k = 2$, $C = 1$ happens in half of the samples. The size of z thus measures how far Y is from commuting with X ; its dependence on $\Delta(Y)$ is shown in the figures. The relation of this quantity to the characterisation of central elements by trace inequalities [?] and to noncommutative L^1 -spaces [?] is a subject for further study.

Listing 3: Experiment 3 implementation

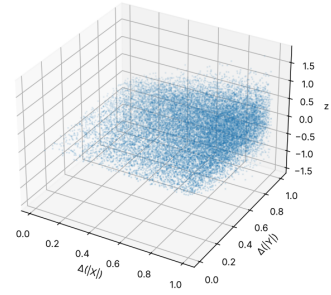
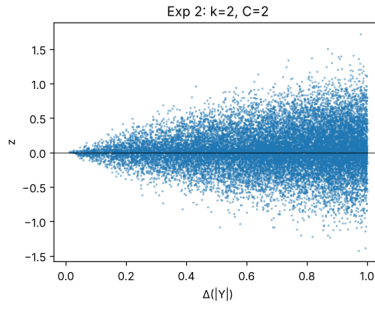
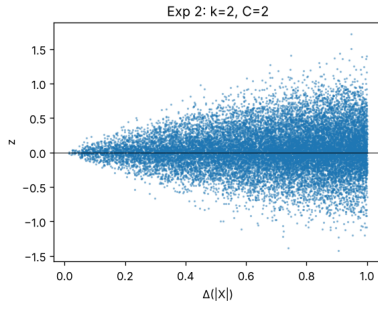
```

1 def experiment3(alg, batch_size, num_batches, device):
2     sa = selfadjoint_sampler(batch_size, device)
3     pos = positive_sampler(batch_size, device)
4     out = []

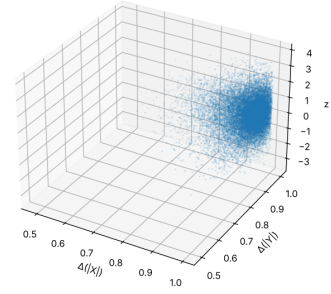
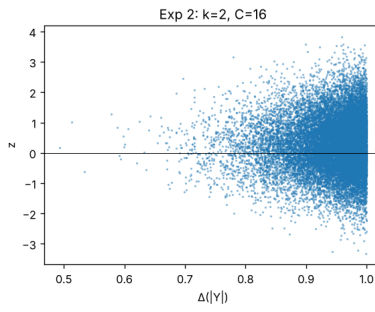
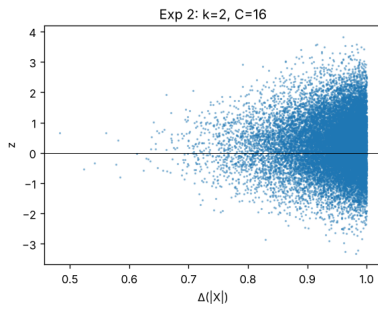
```

(a) $k = 2, C = 1$

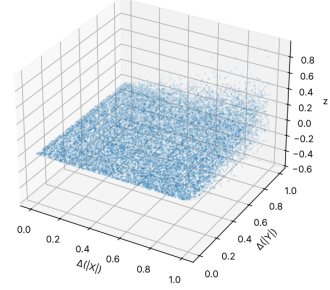
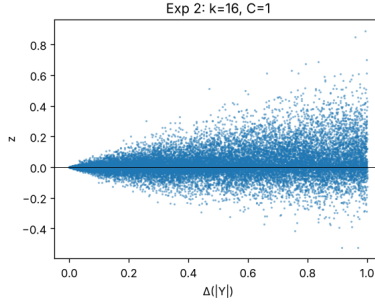
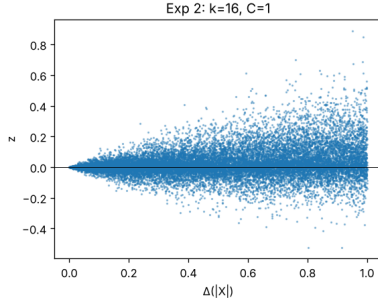


(b) $k = 2, C = 2$

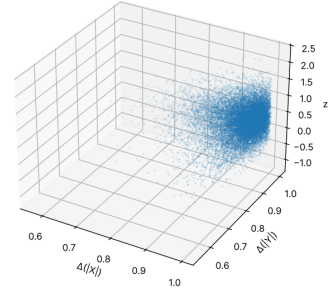
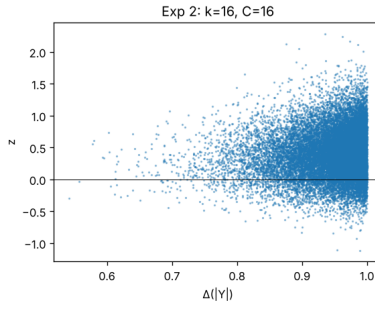
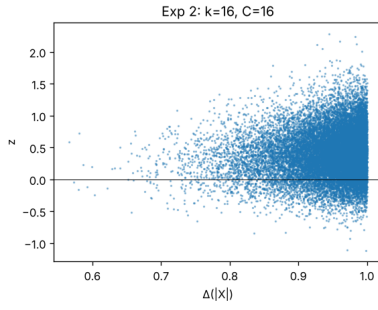


(c) $k = 2, C = 16$

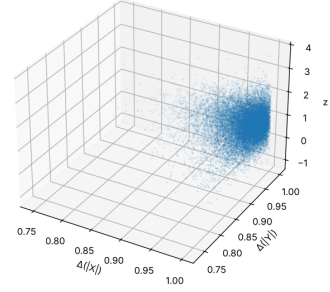
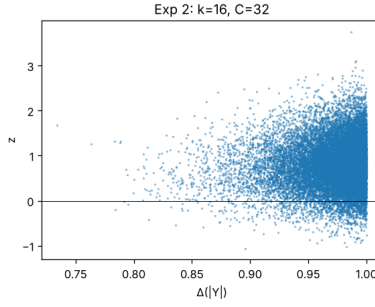
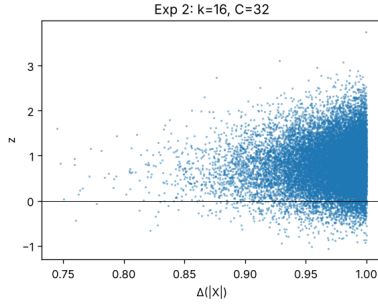
Figure 3: Experiment 2: small dimension ($k = 2$). Left: z vs. $\Delta(|X|)$; middle: z vs. $\Delta(|Y|)$; right: both.



(a) $k = 16, C = 1$



(b) $k = 16, C = 16$



(c) $k = 16, C = 32$

Figure 4: Experiment 2: intermediate dimension ($k = 16$). Left: z vs. $\Delta(|X|)$; middle: z vs. $\Delta(|Y|)$; right: both.

```

5   for _ in range(num_batches):
6       X = alg.operator_from_eigenvalues(sa, batch_size=batch_size, force_self_adjoint=
      True)
7       Y = alg.operator_from_eigenvalues(pos, batch_size=batch_size,
8                                           force_positive=True, force_self_adjoint=True)
9       absX = X.abs()
10      z = (Y @ absX @ Y).trace.real - (Y @ X @ Y).abs().trace.real
11      out.append((absX.michelson_contrast, Y.michelson_contrast, z))
12  return collect(out)

```

6 Limitations and future work

Current limitations include: batched SVD and symmetric eigensolvers lose their GPU advantage already at block size $k = 64$ in our benchmarks (Table ?? and Appendix ??); power iteration converges linearly in the spectral gap, fails to separate eigenvalues $\pm\lambda$ of equal modulus, and its stopping rule controls the change of the estimate rather than the error; automatic differentiation through the operations is not tested; only Type I algebras (direct sums) in finite dimensions are supported. Future work includes support for tensor products, Cauchy-type functional calculus, distributed GPU computing, and mixed-precision SVD. The code is open to contributions.

7 Conclusion

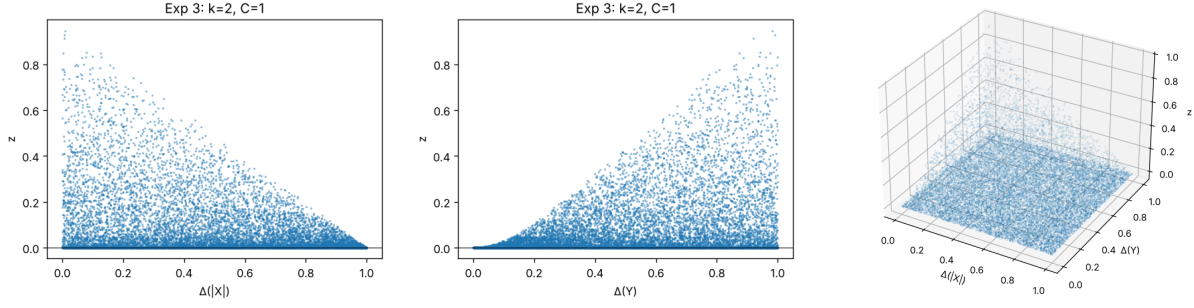
We have presented `torch_vn_algebra`, a PyTorch library for finite-dimensional Type I von Neumann algebras that combines a compact tensor representation, lazy evaluation, flexible random operator generation, and GPU-accelerated functional calculus. The library is validated and benchmarked, and is open-source at https://gitlab.com/a.hobukob/von_neumann_type_i/ (mirror: https://github.com/andre-a-no/von_neumann_type_i).

Data availability

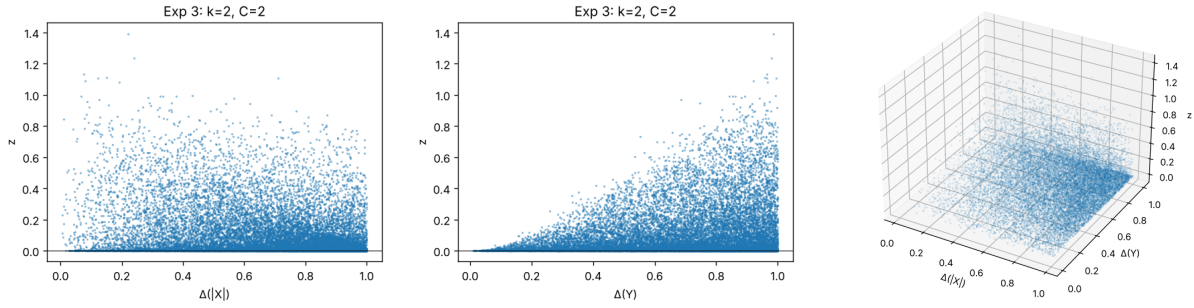
All code, raw data and reproduction scripts are at the above repository.

References

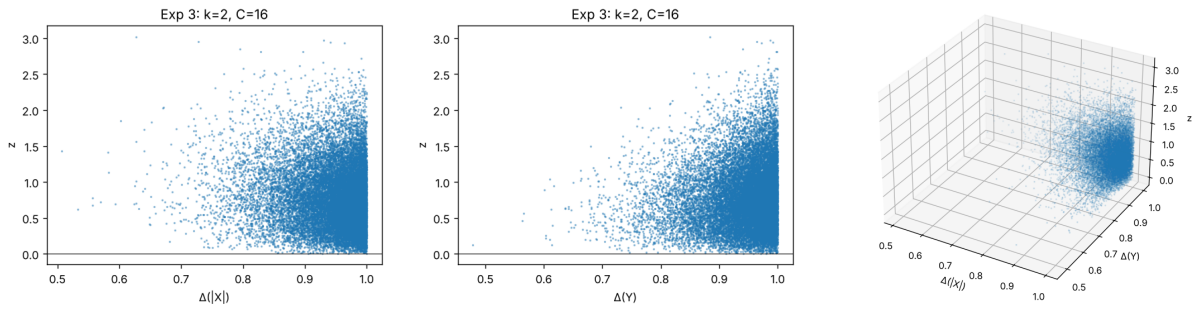
- [1] M. Takesaki, *Theory of Operator Algebras*, Springer, 2003.
- [2] B. Blackadar, *Operator Algebras*, Springer, 2006.
- [3] M. L. Mehta, *Random Matrices*, 3rd ed., Elsevier, 2004.
- [4] M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010.
- [5] L. T. Gardner, “An inequality characterizes the trace”, *Canad. J. Math.* 31 (1979), 1322–1328.
- [6] A. Novikov, O. Tikhonov, “Characterization of central elements of operator algebras by inequalities”, *arXiv:1502.01267* (2015).



(a) $k = 2, C = 1$

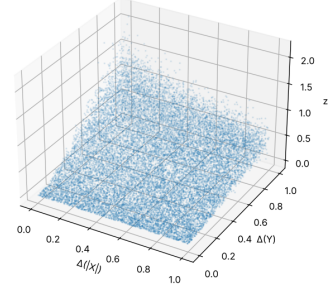
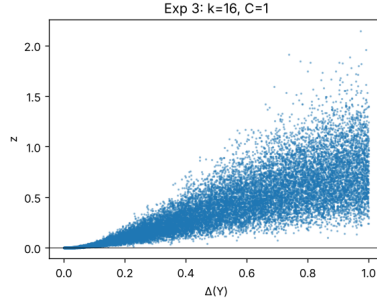
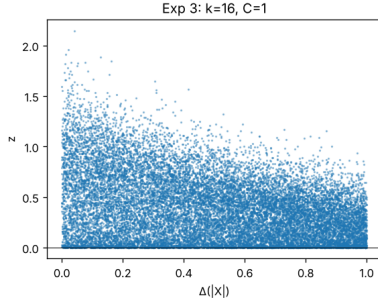


(b) $k = 2, C = 2$

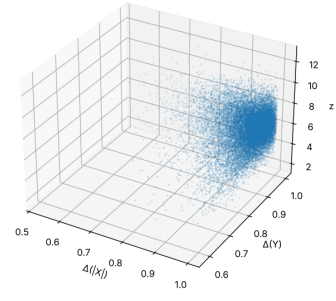
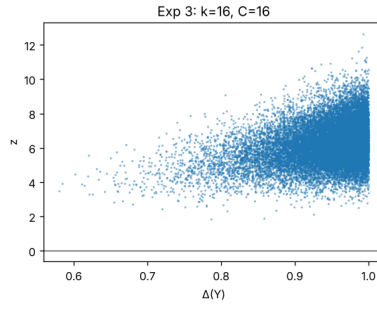
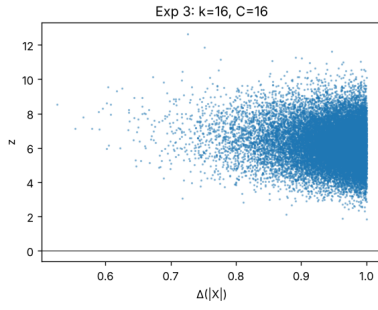


(c) $k = 2, C = 16$

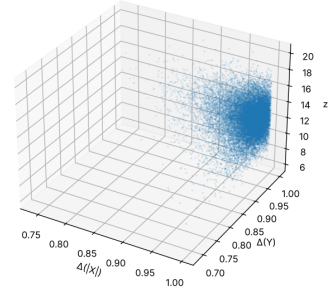
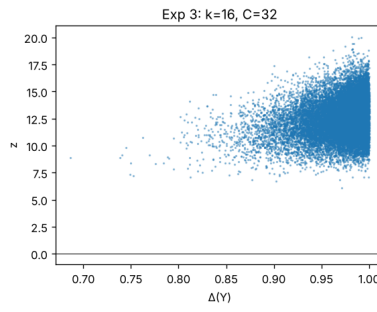
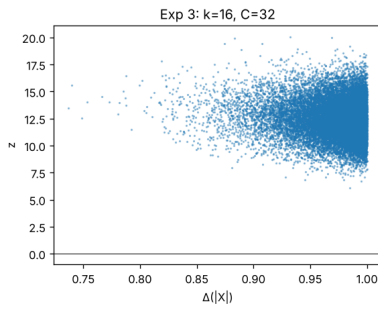
Figure 5: Experiment 3: small dimension ($k = 2$). Left: z vs. $\Delta(|X|)$; middle: z vs. $\Delta(Y)$; right: both.



(a) $k = 16, C = 1$



(b) $k = 16, C = 16$



(c) $k = 16, C = 32$

Figure 6: Experiment 3: intermediate dimension ($k = 16$). Left: z vs. $\Delta(|X|)$; middle: z vs. $\Delta(Y)$; right: both.

- [7] A. Novikov, “ L_1 -space for a positive operator affiliated with von Neumann algebra”, *Positivity* 21 (2017), 359–375.
- [8] D. Petz, J. Zemánek, “Characterizations of the trace”, *Linear Algebra Appl.* 111 (1988), 43–52.
- [9] D. Virosztek, “Connections between centrality and local monotonicity of certain functions”, *J. Math. Anal. Appl.* 453 (2017), 221–226.
- [10] A. A. Michelson, *Studies in Optics*, University of Chicago Press, 1927.
- [11] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, “Generalisation of Michelson contrast for operators and its properties”, *Lobachevskii J. Math.* 45 (2024), 3835–3848.
- [12] A. Novikov, S. A. Abed, I. Nikolaeva, “Generalisation of Michelson contrast for operators”, *arXiv:2010.10130* (2020).
- [13] F. Mezzadri, “How to generate random matrices from the classical compact groups”, *Notices Amer. Math. Soc.* 54 (2007), 592–604.
- [14] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library”, *NeurIPS* 2019.
- [15] C. R. Harris et al., “Array programming with NumPy”, *Nature* 585 (2020), 357–362.
- [16] P. Virtanen et al., “SciPy 1.0: fundamental algorithms for scientific computing in Python”, *Nat. Methods* 17 (2020), 261–272.
- [17] J. R. Johansson, P. D. Nation, F. Nori, “QuTiP 2: A Python framework for the dynamics of open quantum systems”, *Comput. Phys. Commun.* 184 (2013), 1234–1240.
- [18] Qiskit contributors, “Qiskit: An open-source framework for quantum computing”, 2019, <https://qiskit.org>.

A Complete set of speedup heatmaps

`scripts/benchmark.py` generates a heatmap for every operation. Figures ??–?? show the mean GPU/CPU speedup as a function of the block size k (vertical axis) and the number of channels C (horizontal axis); brighter colours (viridis colour map) mean larger speedup, values below 1 mean that the CPU is faster. In the stored run, `lambda_max` was computed by exact diagonalisation of AA^T , `entropy` from normalised singular values and `michelson` from the matrix entries of Gaussian matrices; the current script evaluates them on positive definite matrices from eigenvalues (see `results/benchmark/README.md`).

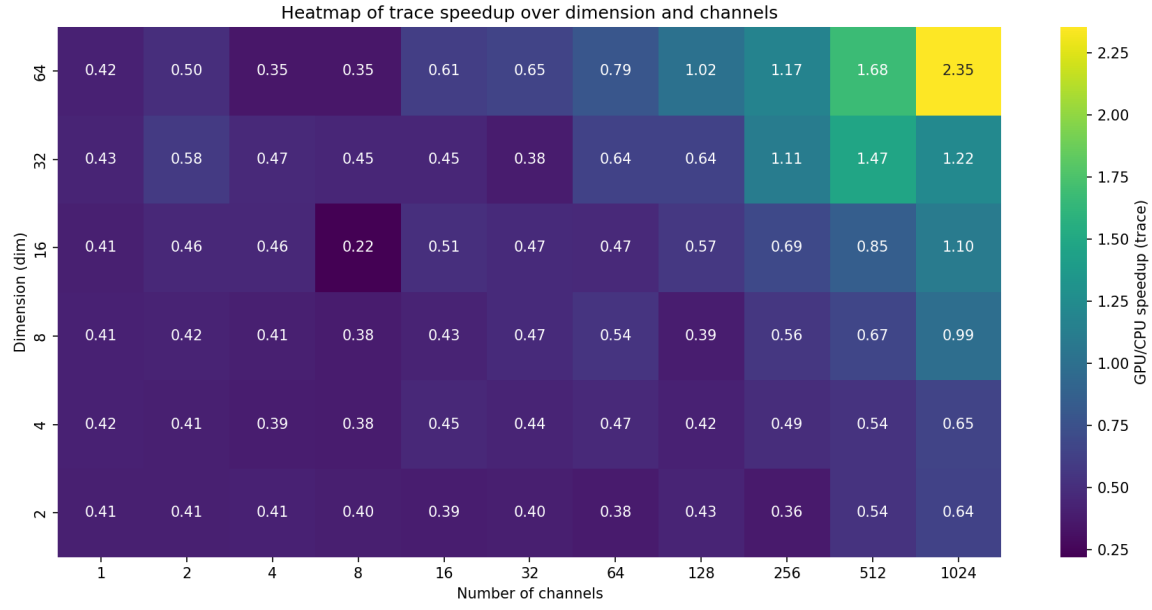


Figure 7: Speedup heatmap: trace.

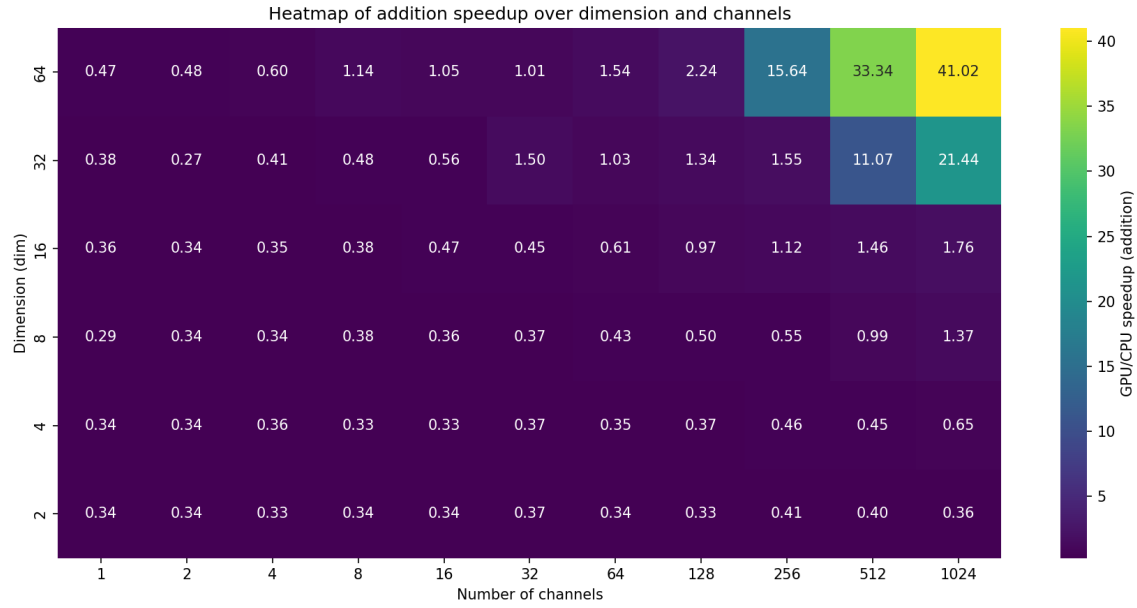


Figure 8: Speedup heatmap: addition.

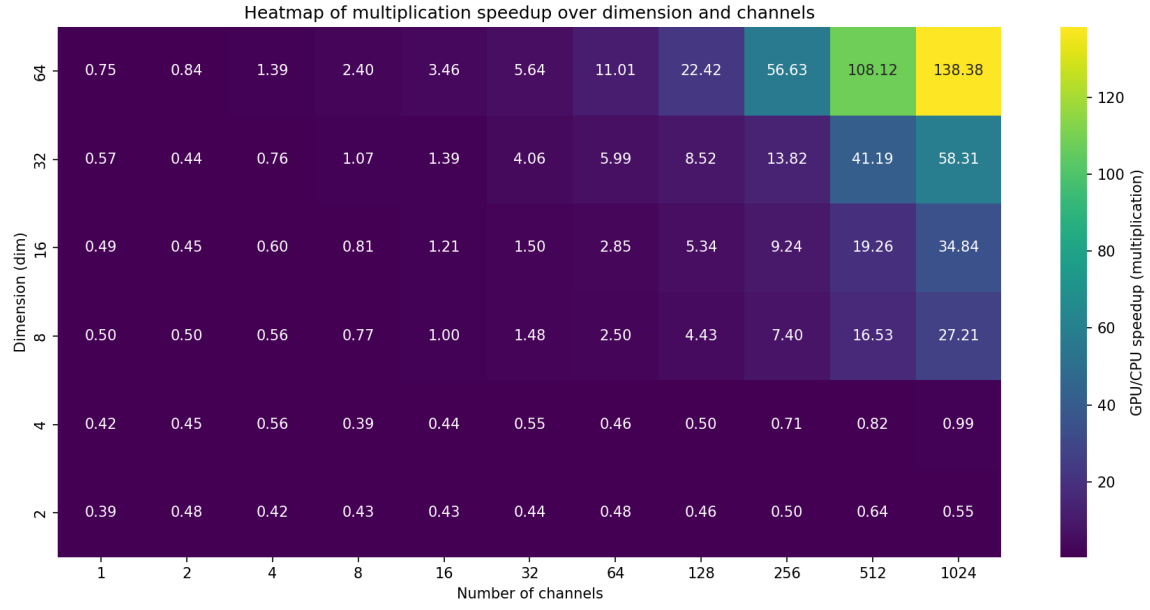


Figure 9: Speedup heatmap: multiplication.

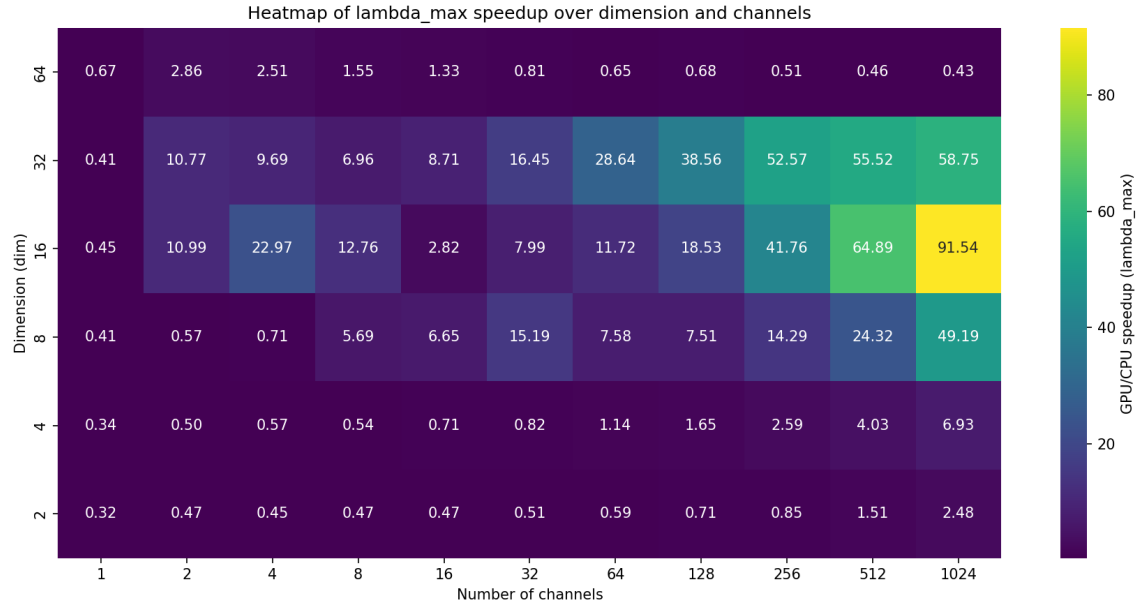


Figure 10: Speedup heatmap: largest eigenvalue.

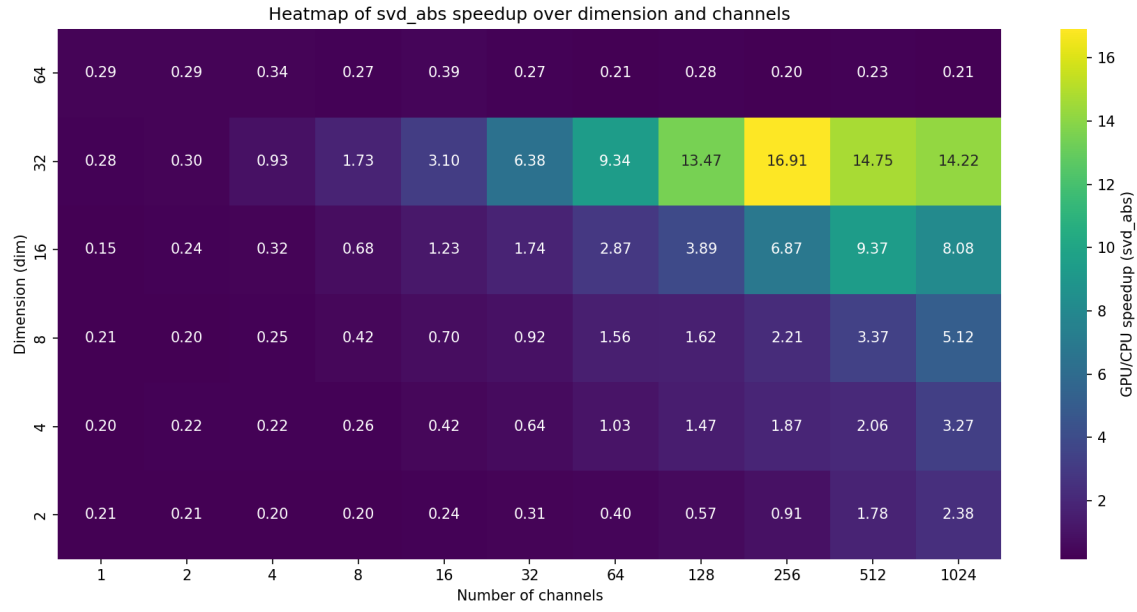


Figure 11: Speedup heatmap: singular values (svd_abs).

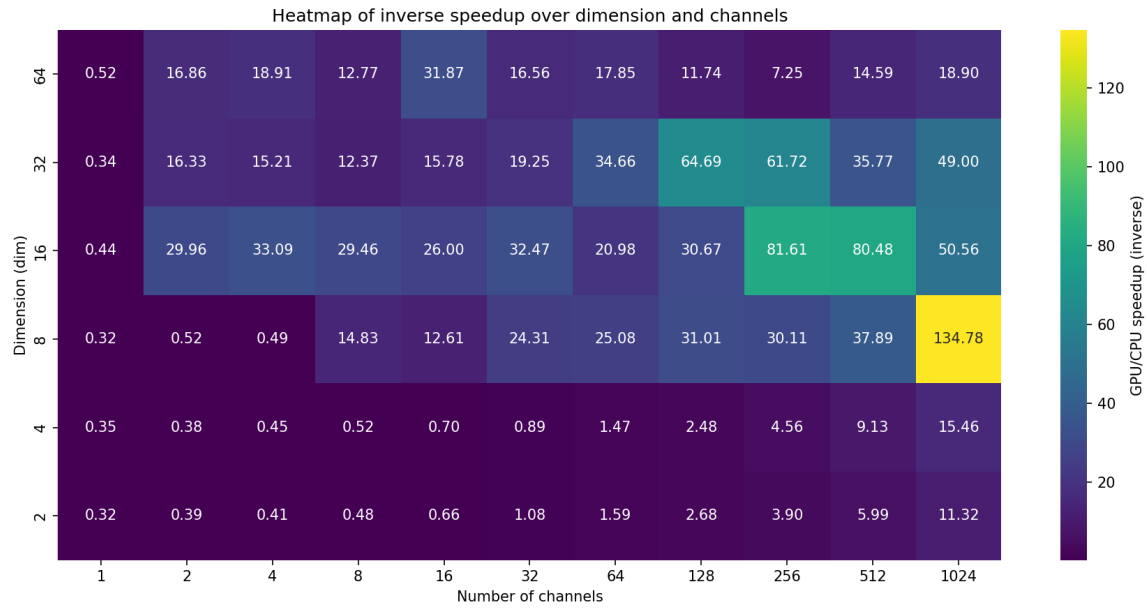


Figure 12: Speedup heatmap: inverse.

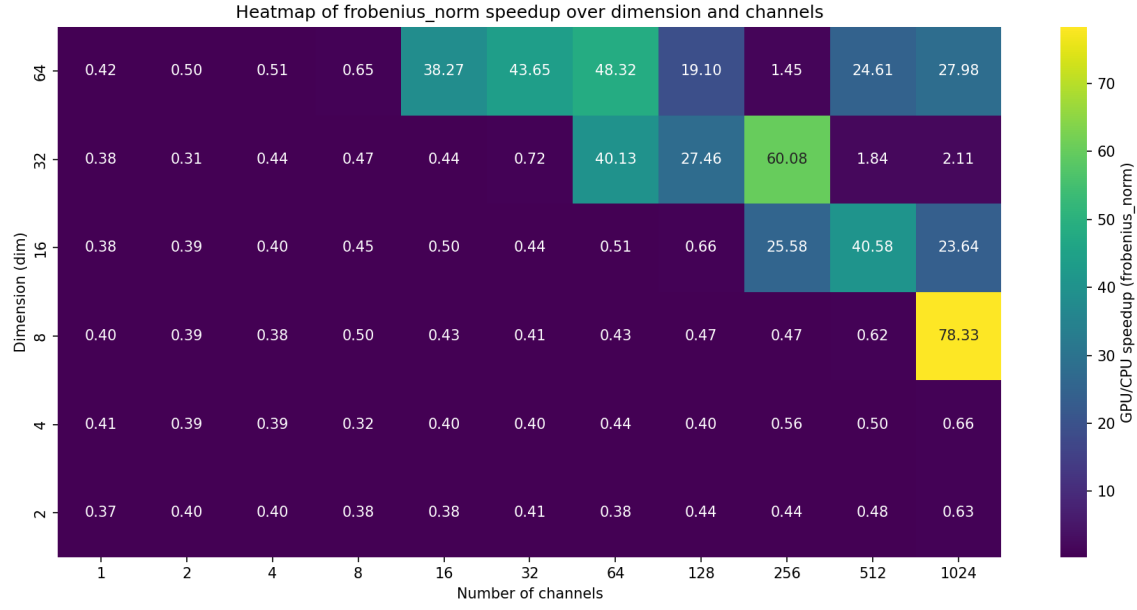


Figure 13: Speedup heatmap: Frobenius norm.

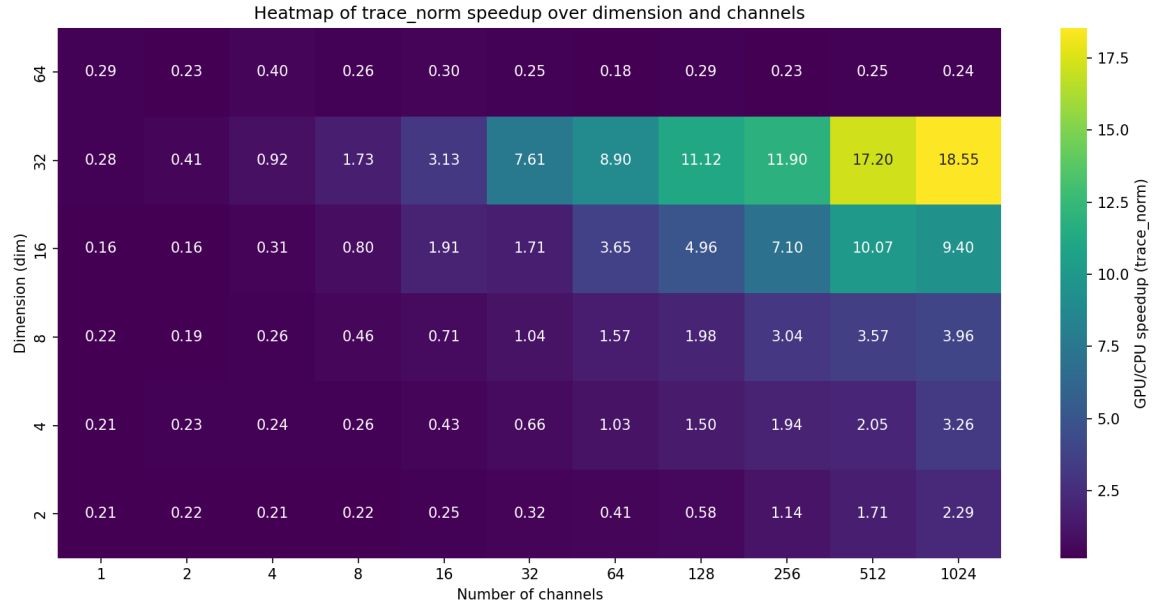


Figure 14: Speedup heatmap: trace (nuclear) norm.

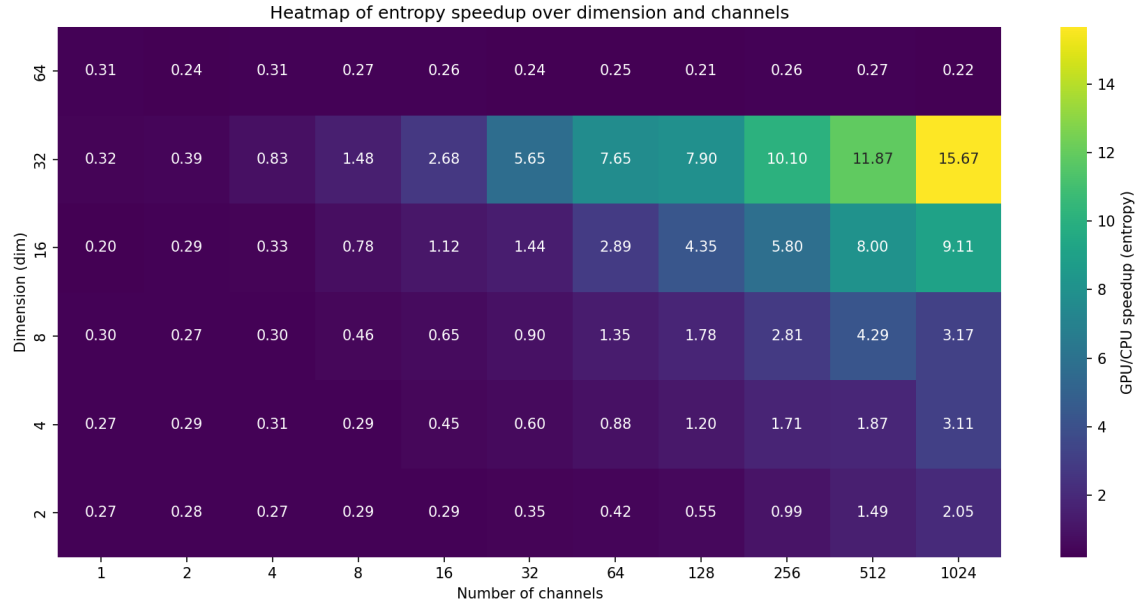


Figure 15: Speedup heatmap: entropy.

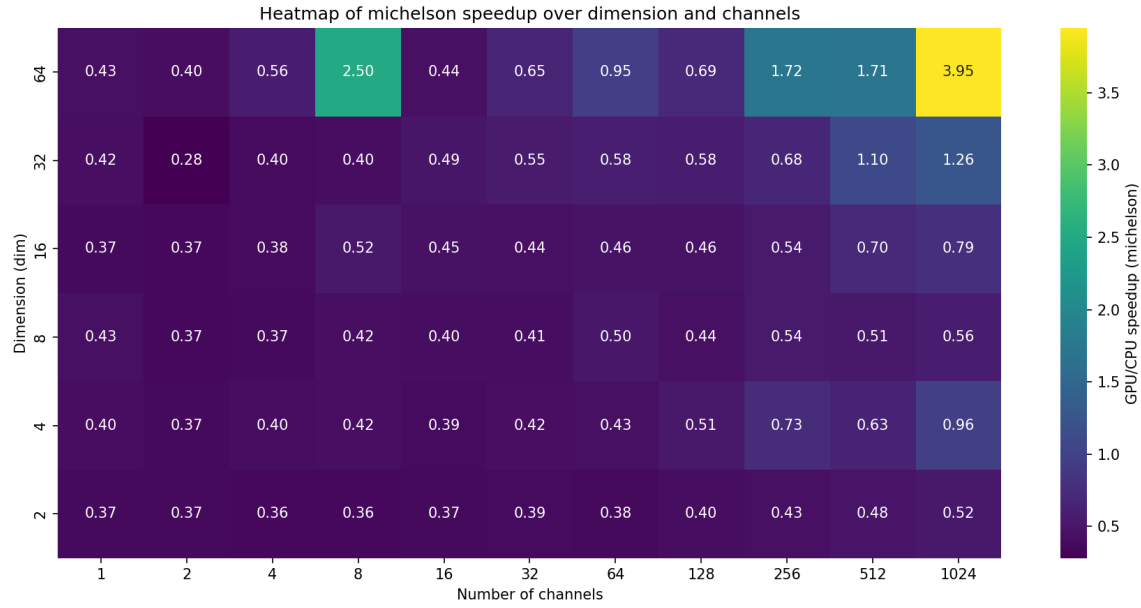


Figure 16: Speedup heatmap: Michelson contrast.